Two Models, One Compiler: Verifier-Centric Coordination for Competition Mathematics in Lean 4

Jacob (jr4682@columbia.edu)
Columbia University

Preprint · Verified Mechanisms take-home submission · August 30, 2026

Abstract

Problem. Two fixed models, `qwen/qwen3.5-flash-02-23` and `openai/gpt-oss-120b`, must jointly solve competition-mathematics problems and prove the results in Lean 4 under per-problem caps of $1.00 and 8 hours. Submissions are scored by the task's Comparator, a program supplied with the kit that rebuilds each submitted file and checks, at the Lean kernel level, that its proofs are valid and its theorem statements equal the challenge's. **Method.** We build a verifier-centric staged controller: deterministic tactics, pooled cross-model sampling, shallow compiler-guided repair, cross-model handoff on repair plateaus, sketch-and-fill decomposition, the kit's own repair chain as a stage, and a *comparator precheck* that re-checks accepted proofs with the scoring Comparator itself instead of trusting the faster development checker alone. **Main controlled result.** On eight held-out problems the controller was never tuned on, at matched 30-minute caps, the raw loops solve 4/8 (Qwen), 3/8 (GPT-OSS), and 4/8 run as a concurrent pair; the same models inside the controller solve 6/8 and 5/8, and the duo 6–7/8. On the kit's 16 problems, by contrast, the raw loops tie the controller arms at the same caps (raw Qwen 9/16, raw GPT-OSS 10/16, arms 8–10/16): the kit's easy tier falls to a bare repair loop, and its value there is composition, zero-cost coverage, and verifier alignment rather than aggregate lift. Pairing the two models adds at most one problem over the better solo arm in either setting, within run-to-run variation, and the extra duo solve traces to a Qwen sampling wave, not to model interaction; per-problem complementarity, not the aggregate, is the primary pairing evidence. **Benchmark audit.** Writing reference proofs surfaced two defective kit statements (one false as formalized, one unscorable under the gate's imports) and a definitional shortcut behind every Putnam pass; the kit's maintainers have since corrected the statements. **Hard instances.** Of the four provable hard problems missed by both original raw baselines, two fall within 30-minute caps (one only with the raw-chain stage, §6.1); multi-hour decomposition runs solved a third; the fourth remains open. A single-model arm given the same controller and a long horizon reproduced two of the three multi-hour solves. Verifier-aligned search infrastructure and search horizon therefore account for more of the gain than model-to-model interaction, with two-family coverage and robustness as smaller, useful additions.

1 Introduction

Formal theorem proving lets a language-model system's output be judged exactly: a Lean 4 proof type-checks against the stated theorem or it does not. The Verified Mechanisms task instantiates this with two fixed, off-the-shelf models served through OpenRouter, Qwen 3.5 Flash (fast, inexpensive, February-2026 training cutoff) and GPT-OSS-120B (slower on the accessible provider tier, cheaper per token, strong at high reasoning effort [28], June-2024 cutoff). It asks for a coordination layer that makes the two jointly solve and prove competition mathematics, and for a scientific account of whether the collaboration helps. Submissions are scored by the kit's *Comparator*: not a part of Lean but a separate scoring program, supplied with the task, that rebuilds the submitted file from scratch and uses the Lean kernel to check that its proofs are valid and that its theorem statements are exactly the challenge's (§2).

We organize the paper around three research questions:

- **RQ1 (scaffold).** How does the controller compare with the kit's raw single-model repair-loop baselines?

- **RQ2 (pairing).** Holding the scaffold fixed, does using both models improve over the better single model, and if so, through what mechanism?
- **RQ3 (hard instances).** What enables solutions on the problems that no short-window configuration solves?

Our design premise is that this task is a coverage problem attached to an exact verifier. The verifier supplies an objective acceptance signal, so no learned or subjective candidate judge is needed, and that shifts value away from dialogue-style coordination, for which compute-matched evidence is weak (§7), and toward candidate diversity, a single consensus decision the verifier cannot check cheaply, and a second set of priors when the first model stalls. What, then, does the second model actually buy? Less than the framing suggests, our results say: coverage on a few problems, one consensus decision, and robustness to a provider outage, while the scaffold and the search horizon carry more of the gain (§5). Contributions: (1) a staged, individually-ablatable controller (Fig. 1) whose comparator precheck runs the kit's own scoring program inside the search loop; (2) a cap-matched design that isolates pairing effects, floor comparisons against the kit's raw baselines, and a 24-problem custom benchmark with Comparator-validated reference proofs and a held-out split; (3) results for RQ1–RQ3 (§5) with a per-problem complementarity analysis and a benchmark audit (§5.4); (4) ablations and robustness (§6), including a ten-mechanism screening.

2 Task and evaluation protocol

Scoring. A problem is solved iff the official *Comparator* accepts every required declaration. The Comparator is a separate scoring program supplied with the kit; it rebuilds the submitted file from a cold start, checks it with the Lean kernel, and demands kernel-level equality between the submitted statements and the challenge statements under a three-axiom whitelist. Numeric answers must be literal decimals, per-problem spend must not exceed $1.00, and the run must finish within the time cap (8 hours at judging).

The development checker differs from the scoring gate. During search, a fast REPL that keeps Mathlib loaded and strips imports checks each candidate; the Comparator builds cold. The two can disagree. A proof the REPL accepts can exceed the Comparator's build-time limit, or its statement can elaborate to a different kernel term under the solution's own imports (§5.4). This gap drives a central design element (§3.3) and a benchmark-validation step (§4).

Budget accounting is fail-closed. If an LLM call fails in a way whose cost cannot be established, the harness permanently stops all further spending for that problem, and the problem scores zero even if a correct proof was saved earlier. The controller therefore runs inside conservative rails: a self-imposed deadline with margin, per-call timeouts, a checkpoint after every improvement, and a degraded mode that finalizes the best saved candidate without further calls once spending is stopped.

Cost structure. Typical per-problem spend is $0.01–$0.10 against the $1.00 cap; the maximum we observed was $0.97, on a budget-exhausted 8-hour run. Wall-clock is usually the binding resource; GPT-OSS latency as served through our provider tier (2–8 minutes per high-effort call) and serial REPL checks dominate it.

Regime	Caps	Problem set	Used for
Raw kit baselines	30 min / $1	kit set (16 public problems)	per-model scaffold-free floor for RQ1
Cap-matched arms	30 min / $1, 2 seeds	kit set	RQ1, RQ2: same controller, model set switched
Development evaluations	20 min / $1	custom dev-16 (§4)	variant selection and mechanism screening (§6)
Held-out checks	20 min / $1	custom held-8 (§4)	promotion checks only; never used for selection
Long-horizon runs	2–8 h / $1	hard subsets of the kit set	RQ3; includes validation runs at exact judge settings

Table 1: Evaluation regimes. The kit set is the task's 16 public problems; the custom dev-16 and held-8 are our own benchmark, described in §4. Every result in §5–§6 is labeled with its regime; comparisons are within-regime unless stated.

Reporting conventions. Repeated identical-configuration runs on the dev-16 vary by up to ±2 problems, driven by three problems whose empirical pass rate across repeated runs is roughly 50%; we therefore report per-problem outcomes wherever a claim depends on them and treat aggregate differences within ±2 as inconclusive. "Cumulative coverage" means solved by any run in the stated regime, not a per-run rate.

3 System

Figure 1 shows the controller: a single anytime escalation ladder whose stages are individually switchable, implemented in `submission/agent.py` over the kit's services. The two models never exchange messages. They interact through shared artifacts (candidate files, error histories, proof skeletons), at a small number of narrow interfaces: independent answer votes (S0.5), pooled sampling (S1), plateau handoff (S3), and alternating sketch duty within decomposition (S4).

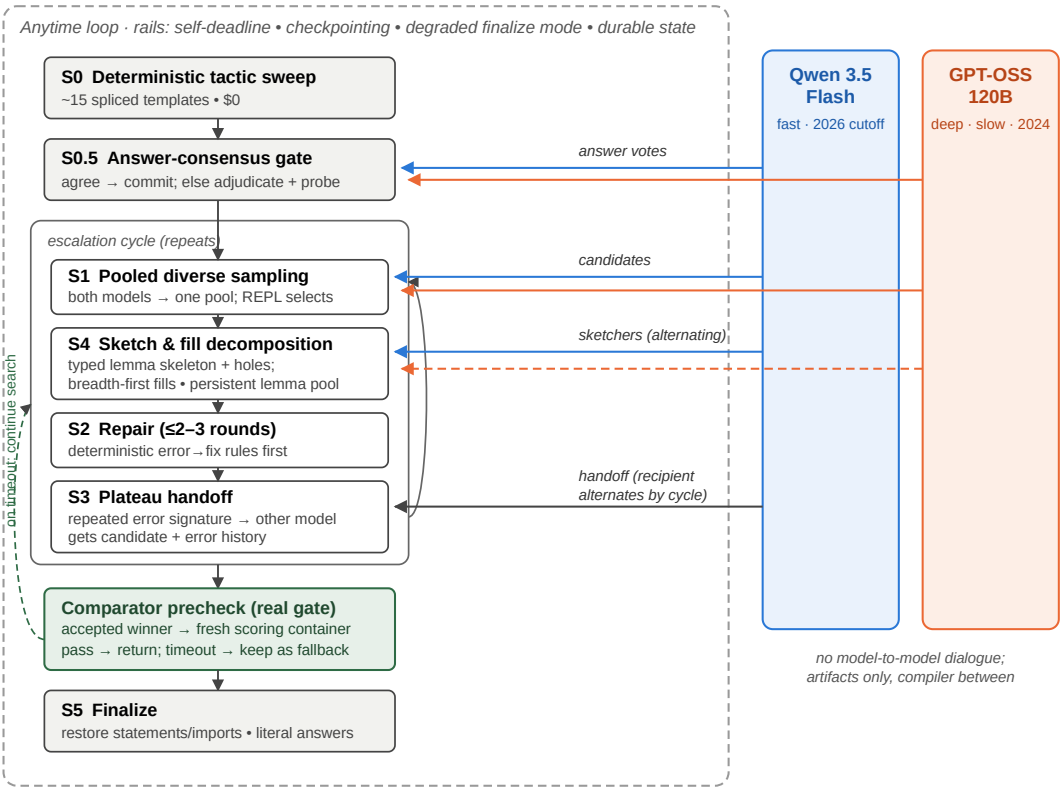

Figure 1: The controller. A staged escalation ladder (left) runs inside the safety rails that fail-closed budget accounting requires; the two models (right) contribute only through artifact-level interfaces. Every REPL-accepted winner must pass a comparator precheck against the real scoring gate before it is returned; a precheck timeout keeps the proof as a fallback while search continues.

3.1 Stages

S0, deterministic sweep ($0). About 15 tactic templates, spliced into the pristine challenge statement, solve the easy third of the kit set before any model is called; since models produce proof bodies and helper lemmas, never statements, the Comparator's statement-equality requirement holds by construction. **S0.5, answer consensus.** For answer-type problems both models independently propose the numeric answer; agreement commits it, disagreement triggers one adjudication round plus inexpensive REPL falsification probes. This is the one decision the verifier cannot check cheaply. **S1, pooled sampling.** Cheap Qwen samples, Qwen samples with reasoning mode enabled (an explicit thinking-token budget; hereafter *Qwen-reasoning*), and GPT-OSS samples enter one deduplicated pool; the

REPL selects. **S1r, raw chain.** The kit baseline's loop, run verbatim once per problem (added last, §6.1). **S2, capped repair.** At most 2–3 error-informed rounds with the model that wrote the candidate, after free deterministic error→fix rewrites (option insertions, misnamed-constant substitution). **S3, plateau handoff.** A repeating error fingerprint routes the candidate plus a condensed error history to the other model, motivated by complementary failure modes on the kit baselines (GPT-OSS repeats a failed attempt; Qwen's attempts vary without converging). **S4, decomposition.** A sketcher (Qwen-reasoning primary, GPT-OSS alternating) writes an informal solution and a Lean skeleton of standalone, explicitly-typed helper lemmas with `sorry` placeholders; invalid skeleton lines are masked instead of discarded [11]; holes are filled breadth-first (every hole gets a cheap attempt before any gets multi-round repair), and proven lemmas persist in a per-problem pool across re-sketches and restarts. **S5, finalize.** Restore pristine statements and imports, enforce literal answers, re-verify, return.

3.2 Window-aware time constants

All stage and model time constants, and the outer loop's cycle budget, scale with the available window, so the same controller runs unchanged at 20-minute development caps and 8-hour judging caps. Fixed constants tuned for one regime silently disable stages in the other (a fixed GPT-OSS call timeout, for instance, leaves no room for decomposition at 20 minutes).

3.3 Comparator precheck

REPL-accepted proofs can exceed the Comparator's cold-build time limit, because kernel-side cost is invisible to the warm checker. The controller therefore runs the same Comparator, in a fresh scoring-style container, on accepted winners before returning them. Acceptance under a wall-clock limit can still depend on machine load, so a precheck timeout does not discard the proof: it is retained as a fallback while search continues for a cheaper one. Section 6.2 quantifies the effect of this component; it involves no second model and is a pure verifier-alignment mechanism. Two further verifier-alignment mechanisms follow from the import-surface gap of §5.4. Every candidate is composed on the challenge's exact import block (invisible to the import-stripping REPL, decisive for the Comparator), and a precheck rejection on a challenge narrower than `import Mathlib` switches the search to core-tactic proofs: one confined repair round, then constrained prompts and linted candidates for the rest of the window.

4 Experimental design

Arms. `solo-qwen`, `solo-gptoss`, and `duo` run the identical controller with only the model set switched, at identical caps and seeds, which isolates pairing effects (RQ2) from scaffold effects (RQ1). In the solo arms the cross-model interfaces degrade to same-model equivalents: the answer gate draws two independent samples from the one model, and plateau handoff restarts the same model with fresh context. The arms are *cap-matched, not compute-matched*. Under identical caps the realized effort differs (per 16-problem run, roughly 121–125 LLM calls for the duo, 334–384 for solo-Qwen, and 17–27 for latency-bound solo-GPT-OSS), so RQ2 compares configurations under equal resource limits, not equal consumed compute. The kit's raw baselines supply the scaffold-free floor per model.

Datasets and development boundary. The kit's 16 public problems informed development throughout (failure analysis, calibration); results on them are development-regime results except where the arms protocol makes the comparison internally controlled. To limit overfitting we authored 24 additional problems in the kit's format (number theory, modular arithmetic, divisor counting, factorial valuations, inequalities, telescoping products): 16 form a development set used for variant selection, and 8 were held out and used only to check promotions (two checks during the project; never used for selection). Three deterministic tactic-cascade entries were mined from archived failed subgoals of earlier runs and are therefore development-derived; they are generic tactic patterns, not problem-specific. The agent never observes reference proofs. Both raw single-model loops were also run on all 24 custom problems at 30-minute caps to supply the benchmark's scaffold-free floors (§5.1).

Benchmark validation. Each custom problem was admitted only after its reference proof was machine-checked in the development REPL. Because this paper's own argument is that REPL acceptance does not imply Comparator acceptance, we also ran all 24 reference proofs through the full Comparator gate; the per-problem results ship with the repository. All 24 passed (cold-build times 33–225 s, none near the gate's limit), so the benchmark's solvability claims hold under the gate that scores submissions.

Metrics and noise. Primary metric: problems solved under the stated regime's caps, with per-problem outcome grids for any claim that depends on composition. Aggregate differences within the observed ±2 dev-16 band are reported as inconclusive. Where attribution matters (which stage or model produced the winning proof), we use the per-problem event logs shipped with each run. The cap-matched kit-set evidence is seven full controller runs (three duo, two per solo arm) and one run per raw loop; the two seeds of each arm were run with successive revisions of the controller, and per-problem rate notes draw on further targeted runs of single problems.

5 Results

5.1 RQ1: the scaffold adds two to three problems in eight on unseen problems; on the kit set it ties the raw loops

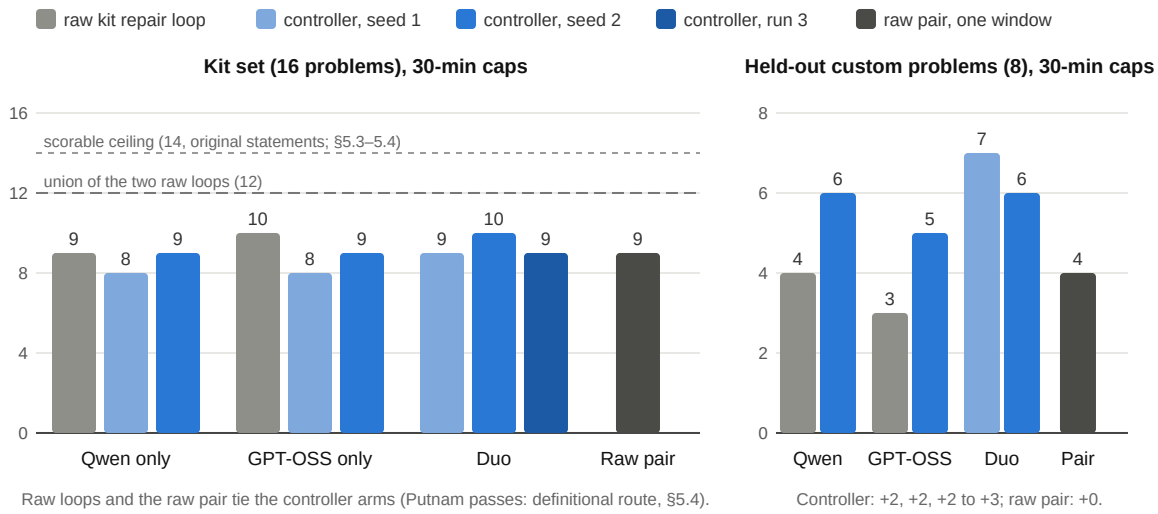

Figure 2: The scaffold effect at matched 30-minute caps. Left, the kit's 16 problems: the raw single-model repair loops tie the controller arms (two seeds per arm, three duo runs). Right, the eight held-out custom problems the controller was never tuned on: the raw loops solve 4 (Qwen) and 3 (GPT-OSS); the same models inside the controller solve 6 and 5, and the duo 7 and 6 over two seeds. Dark grey: both raw loops run concurrently in one window (parallel raw pair). m03 and h03 were solved only with the scaffold.

The controlled gain shows on problems the controller was never tuned on (Fig. 2, right). On the held-out 8 at matched 30-minute caps the raw Qwen loop solves 4/8 and the raw GPT-OSS loop 3/8; the same models inside the controller solve 6/8 (solo-Qwen) and 5/8 (solo-GPT-OSS), and the duo 7/8 and 6/8 over two seeds. Two of the eight (m03, h03) fell only with the scaffold, m03 in most scaffolded runs and h03 once in five; h06 fell in every scaffolded run and in one raw run of three. The development set shows a larger gap (raw 8/16 and 6/16 versus 11–13/16 for the controller at shorter 20-minute caps), but that set informed development, so the held-out 8 is the number to trust. On the kit set, by contrast, the scaffold adds nothing in aggregate (Fig. 2, left). At the arms' 30-minute caps the raw Qwen loop scores **9/16**, parity with the controller's solo-Qwen arm (8–9), and its passes include p09, which no controller arm solved at 30 minutes; the raw GPT-OSS loop scores **10/16**, level with the duo's better seed, passing the seven easy problems, both Putnams (definitional route, §5.4), and p10. What survives at matched caps on the kit set is composition: the free S0 sweep covers 6/16 at $0 where the raw loop spends model calls on every problem; the per-problem sets differ (the raw Qwen loop misses p05, p10, and putnam_2020_a2, which the controller covers); and the controller carries the reliability rails and comparator precheck whose measured value appears at longer horizons (§6.2). The *union* of the two raw loops' passes is 12/16, more than any controller run, but a union of separate runs is not a system. Run as one, two independent raw loops under one budget and one window with the verifier taking the first accepted proof (the least coordination that uses both models; Fig. 2, dark bars), the pair scores 9/16 on the revised statements, level with the controller's 30-minute arms, and 4/8 on the held-out set against the scaffolded arms' 5–7. Concurrent loops realize less than a union of separate runs, and the scaffold's margin on unseen problems

survives this control. The one thing the pair did that no controller arm did at 30 minutes is p09, at its second turn (§6.1).

5.2 RQ2: pairing adds coverage through complementary outcomes; the aggregate gain is small

The duo scored one problem above the best solo arm on both seeds (9 vs. 8; 10 vs. 9) at one-half to one-third of solo-Qwen's cost per run ($0.16–0.17 vs. $0.40–0.45), and in each seed it matched the union of that seed's two solo arms; a third duo run scored 9. Because the aggregate difference sits within the ±2 run-to-run band we observed on repeated identical configurations, we do not treat the +1 as strong evidence by itself; and the parallel raw pair (9/16, §5.1) shows that one window realizes less of the families' complementarity than a union of separate runs suggests. The per-problem grid (Table 2) is the more informative result: the solo arms show complementary per-problem *outcomes* across both seeds, and additional seeds show the headline asymmetries are rate differences, not absolute ones (Table 2 notes). A three-arm replication on the held-out 8 at 30-minute caps points the same way: duo 7/8 and 6/8, solo-Qwen 6/8, solo-GPT-OSS 5/8, raw Qwen 4/8, raw pair 4/8. The duo's extra problem in its better seed (h03, its only solve in five duo runs) came from a Qwen sampling wave in a late cycle. It is a coverage event under the scaffold, not a cross-model artifact: on these problems the scaffold effect dominates.

Problem	Qwen s1/s2	GPT-OSS s1/s2	Duo s1/s2	Note
7 easy/medium problems	✓✓	✓✓	✓✓	solved by every arm, both seeds (6 of them by the $0 sweep)
p07 (least divisible)	✓✓	✗✗	✓✓	solo-GPT-OSS 1/4 across seeds (same-model answer gate chose a wrong path 3 of 4) vs. solo-Qwen 2/2 and duo 2/2
putnam_2018_a1	✗✗	✓✓	✓✓	across seeds: solo-Qwen 2/6, solo-GPT-OSS 3/3, duo 3/3 (duo winners from its own Qwen wave): a rate difference, not a family boundary
putnam_2020_a2	✗✗	✗✓	✗✓	GPT-OSS-side solve in seed 2 for both arms that carry it
p10 (factorial powers)	✗✓	✗✗	✗✗	across all 30-minute attempts p10 passed 2 of 10 (once for solo-Qwen, once for the raw GPT-OSS loop): a low-probability short-cap solve, not an arm difference
3 remaining provable hard problems	✗✗	✗✗	✗✗	p09, rmo_2000_2, rmo_2001_2: out of reach for every arm at 30 minutes (§5.3)
2 unscorable problems	✗✗	✗✗	✗✗	rmo_2000_3, rmo_2000_6: excluded from the pre-revision scorable ceiling (§5.3, §5.4)

Table 2: Per-problem outcomes, cap-matched regime (✓ = Comparator pass; s1/s2 = seeds). Marks show two seeds per arm; the notes report rates over all seeds (n up to 6 per cell). Under more seeds, both headline asymmetries become rate differences rather than absolute family boundaries; the duo inherits the favorable side of each. Putnam checkmarks used a definitional route the revised statements no longer allow (§5.4).

The answer gate's cross-model value appears as a rate on p07: the duo's two-model gate committed the correct answer in 2/2 runs, while solo-GPT-OSS's same-model gate committed a wrong path in 3 of 4 seeds (Table 2).

Two further RQ2 observations come from the long-horizon regime. *Robustness:* during one 14-hour outage of the GPT-OSS provider, a solo-GPT-OSS configuration would have scored zero, while the duo degraded to Qwen-only behavior and kept solving. *Skeleton diversity:* the first full-cap p09 solve grew from a GPT-OSS sketch that Qwen repaired and filled, a structure the solo-Qwen arm did not produce in that run. We report it as a single observed instance, not a systematic effect.

5.3 RQ3: hard instances at short versus long horizons

Six kit problems were unsolved by both original raw baselines. Writing reference proofs for them established that only four are provable as stated. One (rmo_2000_6) is mathematically false as formalized: its second clause asserts a

least value of 20 for a set that contains 10. Another (rmo_2000_3) fails to elaborate under the Comparator's actual import environment, because its file imports only two Mathlib modules, which do not bring `Finset.Ico` into scope, a flaw invisible to the import-stripping REPL. The statement itself is true and provable (our full-Mathlib reference proof compiles), but the Comparator's build of the challenge fails before any solution is considered, so even a correct solution carrying its own `import Mathlib` cannot score. The scorable ceiling is therefore 14/16 under the original statements (certificates in the repository; §5.4 covers the corrected rmo_2000_6). Table 3 gives the per-problem record against the raw loops.

Kit problems (revised statements; 15 scorable)	**raw Qwen**	**raw GPT-OSS**	**controller (best regime)**
p01–p08, easy/medium tier	7	7	8
p09 (IMO 1964)	✓ 4 of 5	✗	✓ 3 of 3 at 8 h (2–7.6 h, $0.10–0.91); solo-Qwen ✓ at 4 h (2 of 2)
p10 (factorial powers)	✗	✓ once	✓ at 2 h (64 min); once at 30 min
rmo_2000_2	✗	✗	✓ 1 of 3 at 8 h ($0.75, 7.4 h); solo-Qwen ✓ at 4 h
rmo_2001_2	✗	✗	✗ one hole short at 4 h and at 12.6 h
both Putnams, rmo_2000_6 (revised)	✗	✗	✗ (24 attempts up to 8 h)
Total (rmo_2000_3 unscorable)	**8**	**8**	**11** (9–11 per run)

Table 3: The kit set as the judges will score it: revised statements (the definitional Putnam passes no longer count), raw loops at their 25-turn limit, controller at its best regime per problem (8-hour caps at judging). At 30 minutes the three columns tie at 8; the controller's margin comes entirely from the long-horizon regime the raw loop cannot enter, and a single 8-hour run is expected to land at 9–11 depending on rmo_2000_2 and p10. Under the original statements the controller's cumulative coverage was 13 of 14 scorable problems.

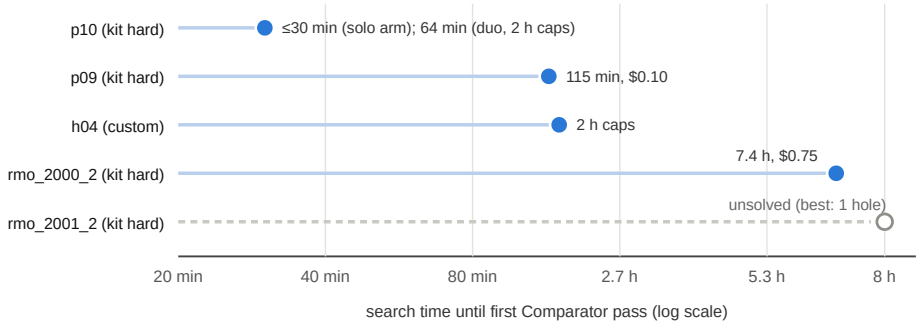

Figure 3: Time until each hard problem first passed the Comparator (hollow marker = unsolved). p10 falls within short caps; every other solve in this tier came at a multi-hour horizon under decomposition; h04 had resisted ~19 short-window configurations.

Short-cap solves in this tier were rare until the raw-chain stage (§6.1): p10 fell within a 30-minute cap under the controller (solo-Qwen arm, one seed; Table 2) and under the raw GPT-OSS loop, and the raw Qwen loop solved p09 in four of five short-window runs (§6.1). The systematic record, however, is at long horizons: the controller's p09 solves are 3-for-3 at 8-hour caps (each requiring the precheck), rmo_2000_2 yielded only after 7.4 hours of accumulated lemma-building, and rmo_2001_2 has not yielded at any horizon tried. A like-for-like raw control now exists: the two raw loops, restarted whenever they exhaust their 25 turns, under one 4-hour window. It found p09 in 16 minutes and nothing else. On rmo_2000_2 and rmo_2001_2 it made 232 and 229 REPL checks over seven to eight chains and then stopped on a candidate the REPL accepted and the Comparator rejected (a theorem-less lemma file; a file importing a module the scoring build lacks): the failure modes the controller's statement guard and import

composition prevent; three guarded reruns, cut short by environment restarts at 12 to 27 minutes, produced no accepted candidate. The raw loops' 4-hour solves are therefore p09 alone, against the controller's rmo_2000_2 at the same horizon, which is evidence that the search organization matters at this tier, not evidence about the models' ceiling.

Two attribution results temper the collaboration reading of Table 3. First, a *solo-Qwen* arm given the same controller and a 4-hour window solved p09 in both runs that tried it (the second verdict fell to a Comparator timeout under load; that proof passes in 51 s on a quiet machine) and rmo_2000_2 in one, so on two of the three multi-hour solves the scaffold and horizon carry the outcome, not the pairing. Second, the h04 winner had been rejected by a precheck timeout and survived only as a retained fallback: a timeout under load is not reliable evidence that a proof is too costly.

5.4 Benchmark audit: defective statements and the import-surface gap

Writing reference proofs for the kit set doubled as an audit of it, and the kit's maintainers revised three statements during our submission window (adopted here). First, the two Putnam problems originally left the answer as a solver-supplied definition; an audit of our event logs found that *every* Putnam pass reported in this paper, in every arm and both raw loops, instantiated that definition with the defining condition itself, a legal shortcut that proves nothing about the mathematics. The revised statements state the answers explicitly, and no arm has solved them since: twenty-four attempts across three arms and caps from 30 minutes to 8 hours all failed. Second, rmo_2000_6 was false as formalized, and the correction matches the witness our falsity certificate exhibits (§5.3), raising the scorable ceiling on the revised set to 15 of 16 (rmo_2000_3 is unchanged). Within 75 minutes of an 8-hour run the duo found a REPL-accepted proof of it, and its comparator precheck rejected the proof: both files build, but the kernel-level statements differ. Import ablations pin the cause. The challenge imports two Mathlib modules; the proof needs the Mathlib tactic library, and importing it makes the identical statement text elaborate to a different kernel term. The challenge-side gap (rmo_2000_3) thus also binds on the solution side whenever a challenge imports less than all of Mathlib. The problem is nevertheless scorable: a reference proof written inside the challenge's import surface, with core tactics and `decide`-driven bounded case analysis in place of `interval_cases`, passes the Comparator in 14 s. The two mechanisms of §3.3 follow: composing candidates on the challenge's imports rescues proofs that added imports they did not need, and the confined core-tactic repair executes end to end but produced no scoring core-only proof for this problem in offline tests (0 of 6 each way), so it is bounded by model capability, not by the machinery.

6 Ablations and robustness

6.1 Mechanism screening on the development set

An initial selection loop promoted two changes (window-proportional time constants and breadth-first hole filling), raising the dev-16 from 11 to 13 with the held-out set stable at 6/8 on both checks. We then screened ten further mechanisms drawn from the literature and from our own failure analysis (Fig. 4). None of the ten reproducibly improved on the tuned default. The only score above the band (verified premise hints, 13/16) fell to 10/16 on its repeat seed, and two mechanisms' triggers fired at most once. The screening is single-toggle and short-window (no interactions, no other horizons), except that skeleton persistence (resuming the best partial decomposition across cycles) was also run at 60-minute and 2-hour caps on the kit's hard four: nothing converted beyond p10, which falls without it. One pattern within it is consistent: every mechanism that lengthened prompts with guidance material scored at or below the default while costing more. We also tested model-to-model dialogue directly: (D1) review-informed repair, where the other model reviews each failing attempt before the author repairs, and (D2) plan dialogue, where the other model critiques the sketcher's informal plan before the sketch is written. At 40-minute caps, where the exchanges reliably fire, D2 scored 12/16 (17 exchanges) and D1 10/16 (24 reviews) against a 40-minute default of 11, at 40–50% higher cost. Neither mechanism converted any problem the default misses.

The pair arm's p09 solve (§5.1) pointed at a mechanism the screening had not covered. The kit's raw loop is one chronological chain of whole-file rewrites at temperature 0.2, each carrying the previous attempt's compiler errors; it solved p09 in four of five short-window runs, while the controller's independent S1 samples and S2 repair rounds produced no accepted p09 candidate in seven. Embedded verbatim as a cycle-1 stage (16 turns, candidates still guarded), it solved p09 in two of three 30-minute runs (turns 13 and 10), the first controller solves of that problem at

that cap; on the kit set it scored 9/16 with no regressions ($0.37 per run against the duo's $0.16: the chain spends up to 16 calls on every unsolved problem), and on the dev-16 at 20-minute caps 13/16 at unchanged cost, the top of the default's band, three problems won by the chain. Loop shape, not model or budget, was the missing ingredient; with the held-out set stable (6/8) the stage became part of the default configuration.

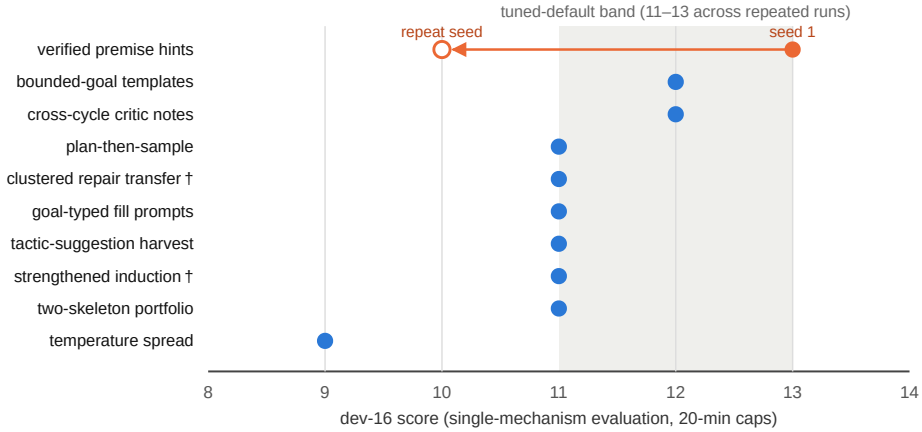

Figure 4: Mechanism screening (development regime). Ten mechanisms, each behind an independent flag, each evaluated with only that flag enabled. † = the mechanism's trigger condition fired at most once during its run, so its score is uninformative about the mechanism itself. The two dialogue mechanisms tested afterwards (12/16 and 10/16 at 40-minute caps) are reported in the text.

6.2 Precheck and reliability

The comparator precheck's contribution is estimated by counterfactual replay of run logs: under a no-precheck return policy one full-cap validation run would have returned two accepted-but-costly proofs and scored 0/4; with the precheck both were rejected internally and a later proof passed. A problem that had failed three times on scoring-time build timeouts was solved once the precheck was in place. The h04 case (§5.3) shows the retain-don't-discard half mattering in the opposite direction. Across all runs of the final configuration, including container restarts and a 14-hour provider outage, no run ended with unaccounted cost.

6.3 Negative results retained

Deeper per-hole reasoning budgets at short windows: no additional structural solves, +16% cost. GPT-OSS inside short-window sampling waves: its latency halves completed cycles.

7 Related work

Repeated sampling against a verifier converts coverage into accuracy [1], and is the baseline any richer mechanism must beat at matched cost. Cross-family candidate pooling captures most ensemble benefit with two models [2], and pays when the selector is strong [3]; in our setting final proof validity is decided mechanically by the Comparator. Compiler-feedback repair helps most when shallow [4, 5, 6, 7]; equal-budget self-repair can lose to resampling [8], and models correct located errors better than they locate them [9], which motivates handing a candidate plus located errors to a *different* model instead of critiquing. Sketch-then-fill decomposition underlies the strongest current systems [10–15], and agentic loops around unmodified general models are now competitive with fine-tuned provers [7, 23, 24]; GPT-OSS-120B appears as the informal reasoner of an open hard-mode pipeline [25] and among the most cost-efficient provers in [26]. Verifier-gated cascades [16] and self-consistency [17] supply the cost-allocation and answer-commitment patterns we adopt. Compute-matched evaluations of multi-agent debate are largely negative [18, 19, 20, 21], and mixture-of-agents synthesis underperforms selection [22, 3]; our own tests of two dialogue mechanisms (§6.1) agree, so dialogue is not part of the default configuration. miniF2F [29] and PutnamBench [30] define the setting; the kit's Putnam items follow PutnamBench's solver-supplied-answer convention (§5.4). AlphaProof [31] marks the frontier with a trained RL prover; we instead coordinate two fixed, unmodified models. Our evaluation methodology follows [27].

8 Limitations

Sample sizes are small (16 kit problems, 24 custom) and the dev-16 varies by ±2 across identical runs, so we avoid significance claims and report per-problem outcomes; the RQ2 aggregate difference (+1 on both seeds) is within that band and the complementarity pattern of Table 2 is the sturdier finding. The long-horizon solo control (4 h) is shorter than the duo's full-cap runs (8 h). Single-seed results (the raw p10 solve) are reported as such. All 24 custom challenges import full Mathlib, so the custom benchmark cannot exhibit the import-surface gap of §5.4, and the long-horizon evidence on the revised statements is one seed. All runs were made on a shared development machine; load-affected runs were rerun under light load or their solutions re-checked on a quiet Comparator (marked as such), and every incident is documented in the experiment log. Finally, these are two fixed mid-scale models: the demonstrated range (full easy/medium tier, several hard instances, olympiad-final tier unsolved) is consistent with published results for comparable backbones [6, 14, 25, 26].

9 Conclusion

When proof correctness is externally verifiable, most of the value of a multi-LLM theorem-proving system in our setting lay in verifier-aligned search infrastructure and complementary candidate coverage, not in conversational coordination. On held-out problems the scaffold raises each model by two problems in eight and the pair by two to three; on the kit set the raw loops tie it at 30 minutes, and there its demonstrated value is zero-cost deterministic coverage, per-problem complementarity, robustness, and its verifier-alignment machinery. Two model families added smaller but real benefits: per-problem coverage the aggregate understates, one consensus decision with a concrete case (p07), an observed cross-family proof structure, and robustness to a provider outage that would have zeroed a single-model configuration. Among the remaining structural instances, systematic success appeared only at long horizons under decomposition; none fell to the short-window prompting variants we screened. Within the mechanisms and budgets we tested, that ordering (controller, then coverage, then horizon) is the paper's main empirical claim; the two dialogue mechanisms added cost without converting any problem.

Reproducibility and disclosure. The repository contains the controller, the custom benchmark with Comparator-validated reference proofs, per-run artifacts and event logs for every result above, and `RUNBOOK.md`; `scripts/judge_check.sh` passes on the submitted configuration. Appendices: `RESEARCH.md`, `PART2.md`, `EXPERIMENTS.md`, `RESEARCH_LOOP.md`. This submission was developed with substantial assistance from Claude (Anthropic): code, experiments, analysis, and writing, as disclosed in the submission form.

References

[1] B. Brown et al. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787, 2024.
[2] F. Vallecillos-Ruiz et al. Wisdom and Delusion of LLM Ensembles for Code Generation and Repair. arXiv:2510.21513, 2025.
[3] A. Maryanskyy et al. When Agents Disagree: The Selection Bottleneck in Multi-Agent LLM Pipelines. arXiv:2603.20324, 2026.
[4] H. Wang et al. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning. arXiv:2504.11354, 2025.
[5] Y. Lin et al. Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction. arXiv:2508.03613, 2025.
[6] A. Ospanov et al. APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning. arXiv:2505.05758, 2025.
[7] Y. Zhou et al. Solving Formal Math Problems by Decomposition and Iterative Reflection. arXiv:2507.15225, 2025.
[8] T. X. Olausson et al. Is Self-Repair a Silver Bullet for Code Generation? arXiv:2306.09896, 2023.
[9] G. Tyen et al. LLMs cannot find reasoning errors, but can correct them given the error location. arXiv:2311.08516, 2023.
[10] A. Q. Jiang et al. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. arXiv:2210.12283, 2022.
[11] C. Cao et al. Reviving DSP for Advanced Theorem Proving in the Era of Reasoning Models. arXiv:2506.11487, 2025.
[12] Z. Z. Ren et al. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. arXiv:2504.21801, 2025.
[13] S. Varambally et al. Hilbert: Recursively Building Formal Proofs with Informal Reasoning. arXiv:2509.22819, 2025.
[14] K. Baba et al. Prover Agent: An Agent-Based Framework for Formal Mathematical Proofs. arXiv:2506.19923, 2025.
[15] L. Chen et al. Seed-Prover: Deep and Broad Reasoning for Automated Theorem Proving. arXiv:2507.23726, 2025.
[16] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.
[17] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171, 2022.
[18] J. Huang et al. Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798, 2023.
[19] A. Smit et al. Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs. arXiv:2311.17371, 2023.
[20] H. Zhang et al. Stop Overvaluing Multi-Agent Debate — We Must Rethink Evaluation and Embrace Model Heterogeneity. arXiv:2502.08788, 2025.
[21] L. B. Kaesberg et al. Voting or Consensus? Decision-Making in Multi-Agent Debate. arXiv:2502.19130, 2025.
[22] W. Li et al. Rethinking Mixture-of-Agents: Is Mixing Different Large Language Models Beneficial? arXiv:2502.00674, 2025.
[23] B. Requena et al. A Minimal Agent for Automated Theorem Proving. arXiv:2602.24273, 2026.
[24] P.-N. Kung et al. LEAP: Supercharging LLMs for Formal Mathematics with Agentic Frameworks. arXiv:2606.03303, 2026.
[25] C. Liu et al. Discover and Prove: An Open-source Agentic Framework for Hard Mode Automated Theorem Proving in Lean 4. arXiv:2604.15839, 2026.
[26] T. Klingner et al. Evaluation of LLMs for Mathematical Formalization in Lean. arXiv:2606.05632, 2026.
[27] S. Kapoor et al. AI Agents That Matter. arXiv:2407.01502, 2024.
[28] OpenAI. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925, 2025.
[29] K. Zheng, J. M. Han, and S. Polu. miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. arXiv:2109.00110, 2021.
[30] G. Tsoukalas et al. PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition. arXiv:2407.11214, 2024.
[31] T. Hubert et al. Olympiad-level formal mathematical reasoning with reinforcement learning. Nature, 2025.